NoC Router

Master Thesis

Qiaosong Jing and Wenqi Li

April 20, 2026

Contents

1 Introduction **2**

2 Functional Description **2**
2.1 Key Features 2
2.2 Parameter Definition 3
2.3 Top Interface Signals 5
2.3.1 Forward and Backward Paths 5
2.3.2 Transfer Semantics 6
2.3.3 Role of `modport` 7

3 Architecture Design **7**
3.1 Microarchitecture 7
3.2 Block Diagram 7
3.2.1 Top-Level Integration 7
3.2.2 Stage 1: RC + VA 9
3.2.3 Stage 2: SA 9
3.2.4 Stage 3: ST 9
3.2.5 Implemented State Variables 10

4 Timing Requirements **10**

5 Verification Plan **10**
5.1 Testbench Strategy 10

1 Introduction

This document specifies the current RTL implementation of a Network-on-Chip (**NoC**) router. The router conducts **flit** movement from source Processing Element (**PE**) to destination PE. The implemented top module is `TOP`, which wraps a three-stage virtual-channel router `ROUTER_3STAGE_VC` with **wormhole switching**, **XY-routing**, and **per-port virtual channels**.

2 Functional Description

2.1 Key Features

- **NoC Topology**: The NoC adopts a 3×3 2 dimensional mesh topology, shown in Figure 1.

Missing figure: `fig1_noc_mesh`

Figure 1: 2D mesh NoC

- **Routing Algorithm**: Dimension-order routing, **XY-routing**: distributed and deterministic routing strategy. Routing result is only determined by the destination and current Cartesian coordinates. The algorithm is shown in Algoritm 1.
- **Wormhole Switching**: The packets are divided into flits (flow control digits), which are all 32-bit in the design. The first flit in a packet (**header**), which contains routing and control information, reserves the channel of each router, the **body** (payload) flits will then follow the reserved channel, and the **tail** flit will later release the channel reservation. The header contains necessary control signals of the packet, whose format is shown in Figure 2. The header is parsed to derive fields, as shown in Table 1. Fields definitions are illustrated

Missing figure: `fig2_header_format`

Figure 2: Header flit format

more detailed in Section 2.2.

- **Virtual Channel**: Helps reduce head-of-line (**HoL**) blocking and allows different traffic flows to share the same physical channel more efficiently.

Algorithm 1 Pseudo code of the XY-routing computation

Definitions:

- cur_x, cur_y: current router coordinates.
- dst_x, dst_y: destination router coordinates.
- $port$: selected output port.

Input:

- cur_x, cur_y: coordinates of current router.
- dst_x, dst_y: coordinates extracted from the header flit.

Output:

- $port$: routing decision toward the next hop.

```
if dst_x > cur_x then
    port ← EAST
else if dst_x < cur_x then
    port ← WEST
else if dst_y > cur_y then
    port ← NORTH
else if dst_y < cur_y then
    port ← SOUTH
else
    port ← LOCAL
end if
return port
```

2.2 Parameter Definition

To improve readability and maintain consistency between the RTL and the architectural description, the main design parameters and user-defined types are summarized below.

Processing Element Position

Each router or PE is identified by its Cartesian coordinate (x, y), where each coordinate ranges from 0 to 2. Since each dimension has three legal positions, the following position type is defined:

$$\texttt{POS} \in \{P0, P1, P2\}$$

with

$$\texttt{POS_NUM} = 3$$

Table 1: Header Flit Fields

Field Name	Bit Slice	Width	Description
DSTX	[31:28]	4	Destination ID.
DSTY	[27:24]	4	Destination ID.
SRCX	[23:20]	4	Source ID.
SRCY	[19:16]	4	Source ID.
FLIT	[15:14]	2	Flit Type.
LEN	[13:10]	4	Packet Length.
VCID	[4:3]	2	Reserved in the current RTL. Next-hop VC selection is carried by the lin
MSG	[2:0]	3	Message Class.

denoting the number of legal coordinate values in one dimension, and

$$\texttt{POS_SIZE} = \lceil \log_2(\texttt{POS_NUM}) \rceil = 2$$

denoting the number of bits required to encode one coordinate.

Thus, a node position in the mesh can be represented by two coordinates, one for the x-dimension and one for the y-dimension.

Message Class

Packets are divided into two message classes:

$$\texttt{msg_class_t} \in \{\texttt{REQ}, \texttt{RESP}\}$$

where:

- `REQ` denotes a request message,
- `RESP` denotes a response message.

This classification is useful when request and response traffic must be distinguished, for example in protocol handling, arbitration, or virtual-channel allocation.

Flit Type

Each packet is divided into one or more flits. The flit type is defined as

$$\texttt{flit_type_t} \in \{\texttt{FLIT_HEAD}, \texttt{FLIT_BODY}, \texttt{FLIT_TAIL}, \texttt{FLIT_HEADTAIL}\}.$$

The meaning of each type is as follows:

- `FLIT_HEAD`: the head flit, which carries routing and control information,
- `FLIT_BODY`: an intermediate flit carrying payload data,
- `FLIT_TAIL`: the tail flit, which marks the end of a packet,
- `FLIT_HEADTAIL`: a single-flit packet that acts as both head and tail.

Port Definition

Each router has five ports corresponding to one local connection and four cardinal directions:

$$\texttt{port_t} \in \{\texttt{LOCAL}, \texttt{NORTH}, \texttt{SOUTH}, \texttt{WEST}, \texttt{EAST}\}.$$

These ports are used for both input and output directions in the router. The total number of ports is

$$\texttt{PORT_NUM} = 5.$$

Accordingly, the required encoding width is

$$\texttt{PORT_SIZE} = \lceil \log_2(\texttt{PORT_NUM}) \rceil = 3.$$

Virtual Channels

To support virtual-channel flow control, each physical input port contains multiple virtual channels (IVCs and OVCs). Request and response message class each owns 2 VC respectively. The number of VCs per message class is

$$\texttt{VC_PER_CLASS} = 2.$$

Thus the number of VCs at each port is

$$\texttt{VC_PER_PORT} = 2 \times 2 = 4.$$

The number of bits required to index a virtual channel at an output is

$$\texttt{VC_PRT_SIZE} = \lceil \log_2(\texttt{VC_PER_PORT}) \rceil = 2.$$

With five ports, the total number of virtual channels is

$$\texttt{VC_NUM} = 4 \times 5 = 20.$$

Therefore, the number of bits required to index a virtual channel is

$$\texttt{VC_SIZE} = \lceil \log_2(\texttt{VC_NUM}) \rceil = 5.$$

The implemented router uses

$$\texttt{FIFO_DEPTH} = 4$$

buffer entries for each input VC. In addition, the output-credit counter is initialized by

$$\texttt{DOWNSTREAM_VC_DEPTH} = \texttt{FIFO_DEPTH} + 2$$

by default, in order to account for the deeper round-trip induced by the pipelined `SA` and `ST` stages.

Summary

Table 2 summarizes the key parameters used in the design.

2.3 Top Interface Signals

The current top-level interface is defined by the SystemVerilog `interface router_vc_flit_if`. Each physical port carries flit data, packet-boundary flags, the selected virtual-channel identifier on the link, and a per-VC returned-credit bitmap. The overview of the interface is shown in Table 3.

2.3.1 Forward and Backward Paths

Every NoC router could be both TX and RX, as shown in Figure 3. According to the direction of the signals, the interface can be divided into two logical groups:

- **Forward path:** `flit_data`, `is_head`, `is_tail`, `vc_id`, and `valid`. These signals are driven by the transmitter and consumed by the receiver.
- **Backward path:** `ready` and `credit_return`. These signals are generated by the receiver and sent back to the transmitter for flow control.

Table 2: NoC parameter definitions

Parameter / Type	Value / Members	Description
POS	{P0, P1, P2}	Coordinate value in one dimension
POS_NUM	3	Number of legal positions per dimension
POS_SIZE	2	Bit width of one coordinate
DATA_WIDTH	32	Width of one flit
msg_class_t	{REQ, RESP}	Message class
flit_type_t	{HEAD, BODY, TAIL, HEADTAIL}	Flit category
port_t	{LOCAL, NORTH, SOUTH, WEST, EAST}	Router port direction
PORT_NUM	5	Number of router ports
PORT_SIZE	3	Bit width of port index
VC_PER_CLASS	2	Number of VCs owned by one message class at one port
VC_PER_PORT	4	Number of virtual channels at one physical port
VC_PRT_SIZE	2	Bit width of one per-port VC index
VC_NUM	20	Total number of VCs in the whole router
VC_SIZE	5	Bit width of one global VC index
FIFO_DEPTH	4	Depth of each input VC FIFO

2.3.2 Transfer Semantics

A flit is transferred successfully only when both `valid` and `ready` are high in the same clock cycle. In that case, the receiver accepts the flit, the associated control flags (`is_head`, `is_tail`) specify its role within the packet, and `vc_id` identifies which virtual channel is occupied on the link for this transfer:

- `is_head = 1`: the flit is the first flit of a packet,
- `is_tail = 1`: the flit is the last flit of a packet,
- `is_head = 1` and `is_tail = 1`: the packet consists of a single flit,
- `is_head = 0` and `is_tail = 0`: the flit is a body flit.

Table 3: NoC Top module ports

Signal Name	Source	Width	Description
flit_data	TX	DATA_WIDTH	Header, body or tail flit data.
is_head	TX	1	Header flit flag.
is_tail	TX	1	Tail flit flag.
vc_id	TX	VC_PRT_SIZE	Output VC index used on the current physical link.
valid	TX	1	Flit data is valid.
ready	RX	1	Receiver is ready.
credit_return	RX	VC_PER_PORT	Returned credit bitmap from the downstream router.

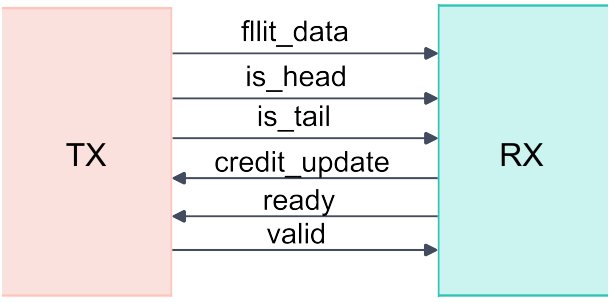

Figure 3: Router Interface

For the returned-credit path, `credit_return[k] = 1` indicates that the downstream router has consumed one flit from its input VC k in the same cycle, so the upstream router can increment the credit counter of the corresponding output VC.

2.3.3 Role of `modport`

The interface defines two `modport`s to constrain signal directions depending on whether a module acts as a transmitter or a receiver:

- `tx` **modport:** used by the sending side. It outputs `valid`, `flit_data`, `is_head`, `is_tail`, and `vc_id`, while receiving `ready` and `credit_return`.
- `rx` **modport:** used by the receiving side. It inputs `valid`, `flit_data`, `is_head`, `is_tail`, and `vc_id`, while driving `ready` and `credit_return`.

This separation improves readability and prevents illegal signal driving in module instantiations.

3 Architecture Design

3.1 Microarchitecture

The implemented router adopts the three-stage pipelined organization **RC+VA | SA | ST**, as shown in Figure 4. This corresponds to the organization of Section 9.5.2 in the referenced microarchitecture literature: routing computation (`RC`) and virtual-channel allocation (`VA`) are executed in the first stage, switch allocation (`SA`) in the second stage, and switch traversal (`ST`) in the third stage.

3.2 Block Diagram

The block diagram is depicted in Figure 5. The figure is conceptual: in the current RTL, `TOP` instantiates a single integrated module `ROUTER_3STAGE_VC`, while the functionality is still naturally decomposed into input virtual channels, route computation plus VC allocation, switch allocation, and switch traversal.

3.2.1 Top-Level Integration

The current top-level wrapper is

```
TOP(CLK, RSTn, IFLIT[PORT_NUM], OFLIT[PORT_NUM])
```

with configurable router coordinates `THISX` and `THISY`. `TOP` only instantiates the current implementation module `ROUTER_3STAGE_VC`. Therefore, the architectural state described below is the actual state held inside that module.

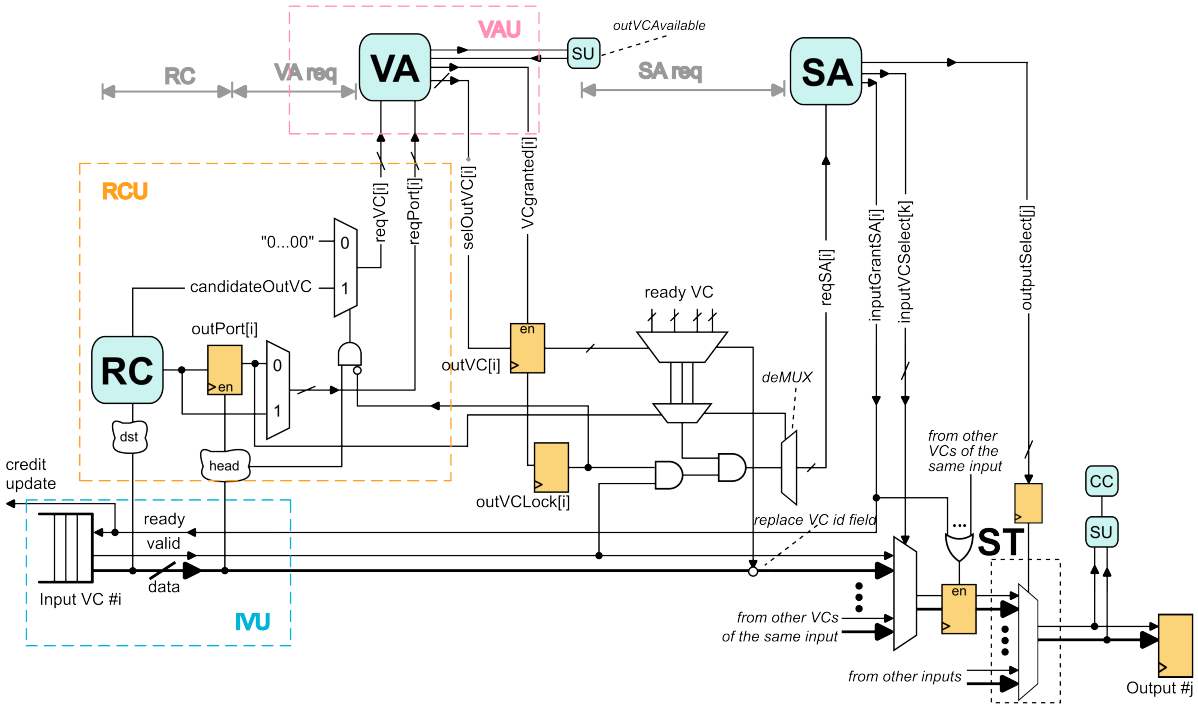

Figure 4: Router Microarchitecture

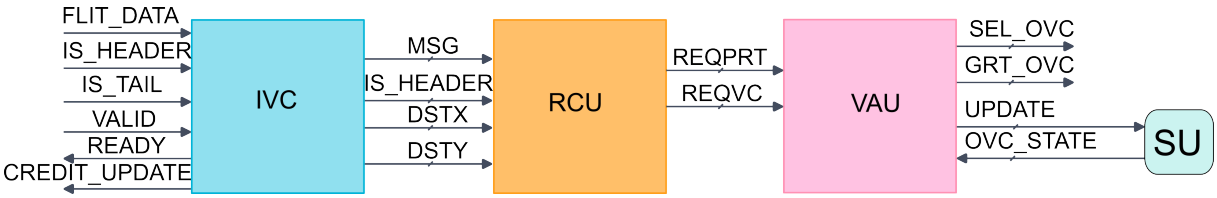

Figure 5: Router Block Diagram

3.2.2 Stage 1: RC + VA

Each physical input port owns `VC_PER_PORT = 4` synchronous input VC FIFOs. The FIFO output is always visible at the front of the queue, which allows the router to inspect the front flit without dequeuing it.

- If the front flit is a head flit and the input VC is not locked by a previously allocated packet, `DSTX`, `DSTY`, and `MSG` are decoded from the header.
- The routing result is produced by the XY-routing function and stored as the requested output port.
- In the same cycle, the router allocates one free output VC from the requested output port and from the appropriate message-class subset. In the current RTL, request packets use output VCs 0 and 1, while response packets use output VCs 2 and 3.
- The allocation result is written into the first-stage pipeline registers `s1_outport_q` and `s1_ovc_q`. At the same time, the packet-level lock state `ivc_lock_{valid,outport,ovc}_q` is established so that following body and tail flits reuse the same route and output VC.

3.2.3 Stage 2: SA

Stage 2 performs switch allocation. For each input port, at most one local VC can win. For each output port, at most one requester can win. The current implementation reuses the separable input-first allocator already present in the RTL.

- Only input VCs whose first-stage register is valid are allowed to request SA.
- The winner dequeues one flit from the selected input VC FIFO.
- The winning flit is captured by the second-stage data registers `s2_{flit_data,is_head,is_tail,outport`
- One downstream credit is consumed from the selected output VC when the flit advances from Stage 2 to Stage 3.
- If the dequeued flit is a tail or head-tail flit, the corresponding input-VC packet lock is cleared at the end of Stage 2, which allows the next packet in the same input VC to perform `RC+VA` in a later cycle.

3.2.4 Stage 3: ST

Stage 3 drives the link output. The flit stored in `s2_*` traverses the crossbar and appears on `OFLIT[out_port]` with the previously allocated `vc_id`.

- A Stage-3 flit leaves the router when `valid` and `ready` are both asserted on the selected output link.
- If the transmitted flit is a tail or head-tail flit, the corresponding output VC busy bit is cleared, releasing the virtual channel for another packet.
- Returned credits from the downstream router increment the per-output-VC credit counters `ovc_credit_q`.

3.2.5 Implemented State Variables

The current integrated implementation replaces the earlier idea of a separate `VAU` and `SU` top-level block with explicit in-module state registers:

- **First-stage registers**: `s1_valid_q`, `s1_outport_q`, and `s1_ovc_q`.
- **Input-VC packet lock**: `ivc_lock_valid_q`, `ivc_lock_outport_q`, and `ivc_lock_ovc_q`. These hold the wormhole reservation for body and tail flits.
- **Output-VC occupancy**: `ovc_busy_q`. A bit is asserted when an output VC has been allocated to an in-flight packet and is not yet released by its tail flit.
- **Output-VC credits**: `ovc_credit_q`. These counters track how many downstream buffer slots are currently available for each output VC.
- **Second-stage data registers**: `s2_valid_q`, `s2_flit_data_q`, `s2_is_head_q`, `s2_is_tail_q`, `s2_outport_q`, and `s2_ovc_q`.

4 Timing Requirements

- **Clock Frequency**: Target frequency of MHz on nm process node.
- **Latency**: In the no-contention case, the head flit requires three router stages, namely `RC+VA`, `SA`, and `ST`, from the time it becomes visible at the front of an input VC until it is transmitted on the output link.
- **Throughput Target**: Once a packet has reserved an input VC and an output VC, body and tail flits can continue advancing without re-running VC allocation, subject to switch-allocation success and downstream readiness.

5 Verification Plan

5.1 Testbench Strategy

- **Implemented testbench**: `TB_ROUTER_3STAGE_VC.sv`.
- **Base Test**: Inject a three-flit request packet into `LOCAL/VC0` of a router located at $(P1, P1)$ and verify that the packet is routed to `EAST`.
- **Wormhole continuity**: Verify that the body and tail flits use the same output VC that was allocated to the head flit.
- **Tail release**: After the first packet finishes, inject a following head-tail request packet in the same input VC and verify that it can acquire a new reservation and be routed independently.
- **Observed result**: The current testbench passes under `xvlog`, `xelab`, and `xsim`.